

Universidad Europea Miguel de Cervantes

Escuela Politécnica Superior

Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Detección de Phishing en Correos Electrónicos

Autor: Alejandro Villarrubia García

Tutor: Carmelo González García

2026

Resumen

El phishing se ha consolidado como una de las ciberamenazas más persistentes y dañinas del panorama actual de la ciberseguridad. Este Trabajo Fin de Grado aborda el diseño, la implementación y la evaluación de una aplicación web local para detectar phishing en correos electrónicos. La solución combina un sistema heurístico explicable con un clasificador neuronal basado en TF-IDF y perceptrón multicapa (MLP). Desarrollada en Python con Streamlit, permite analizar texto pegado, archivos .eml y mensajes importados mediante la API de Gmail, además de entrenar y evaluar modelos en español e inglés. El análisis puede ejecutarse dentro de la aplicación mediante módulos reutilizables y, de forma opcional, exponerse a la extensión de Gmail o a clientes locales mediante una API HTTP; ninguna de esas interfaces envía el correo a un servicio externo. La memoria estudia el fenómeno del phishing desde su base psicológica y su evolución reciente, revisa el estado del arte en machine learning y deep learning, y presenta los resultados obtenidos durante la validación del prototipo. Los resultados confirman que combinar reglas explicables y aprendizaje supervisado constituye una base funcional y extensible para la detección de este tipo de amenazas.

Palabras clave: phishing, ciberseguridad, ingeniería social, machine learning, deep learning, detección de correo electrónico.

Abstract

Phishing has become one of the most persistent and damaging threats in today's cybersecurity landscape. This Bachelor's Thesis presents the design, implementation and evaluation of a local web application for detecting phishing emails. The solution combines an explainable heuristic system with a neural classifier based on TF-IDF and a multilayer perceptron (MLP). Built in Python with Streamlit, it can analyse pasted text, .eml files and messages imported through the Gmail API, as well as train and evaluate Spanish and English models. The analysis runs through reusable local modules and can optionally be exposed to the Gmail extension or local clients through an HTTP API; email content is not sent to an external service. The document examines phishing from its psychological foundations and recent evolution, reviews the state of the art in machine learning and deep learning, and reports the results obtained during prototype validation. The results show that combining explainable rules with supervised learning provides a functional and extensible basis for detecting this type of threat.

Keywords: phishing, cybersecurity, social engineering, machine learning, deep learning, email detection.

Agradecimientos

Quiero expresar mi agradecimiento a mi tutor, Carmelo González García, por su orientación, sus comentarios y su apoyo constante durante la realización de este trabajo. Agradezco también a mi familia y amigos su comprensión y ánimo a lo largo de toda la carrera, así como a todas las personas que de una u otra forma han contribuido a que este proyecto fuese posible.

Índice de figuras y tablas

Figuras

Figura 2.1: Diagrama de flujo swimlane de las fases de un ataque de phishing.

Figura 2.2: Vectores iniciales en brechas analizadas por el DBIR 2025 (n=9.891).

Tablas

Tabla 2.1: Indicadores de contexto del phishing y las brechas (2025).

Tabla 2.2: Comparativa de modelos.

Tabla 2.3: Controles por fase.

Tabla 2.4: Aplicación de los principios de Cialdini en campañas de phishing.

Tabla 3.1: Comparativa de técnicas clásicas de machine learning.

Tabla 3.2: Comparativa de técnicas de deep learning y transformers.

Tabla 4.1: Planificación temporal del proyecto.

Tabla 4.2: Herramientas y tecnologías utilizadas.

Abreviaturas

API: Interfaz de operaciones que permite a un cliente local enviar datos al backend mediante un contrato HTTP; Gmail y Telegram son integraciones externas distintas.

AiTM: Adversary-in-the-Middle; ataque que se sitúa entre el usuario y el servicio legítimo para interceptar la autenticación.

BEC: Business Email Compromise; fraude que suplanta a directivos o proveedores.

BERT: Bidirectional Encoder Representations from Transformers; modelo de lenguaje pre-entrenado.

CNN: Red neuronal convolucional (Convolutional Neural Network).

CSV: Formato de datos separados por comas (Comma-Separated Values).

DBIR: Data Breach Investigations Report de Verizon.

DKIM: DomainKeys Identified Mail; firma criptográfica de los correos del dominio remitente.

DL: Deep Learning (aprendizaje profundo).

DMARC: Domain-based Message Authentication, Reporting and Conformance.

EML: Formato de archivo de correo electrónico estándar.

F1: Media armónica entre precisión y recall; métrica de evaluación de clasificadores.

GAN: Red generativa antagónica (Generative Adversarial Network).

GenAI: Inteligencia artificial generativa.

GPT: Generative Pre-trained Transformer.

GRU: Gated Recurrent Unit; tipo de red neuronal recurrente.

IA: Inteligencia artificial.

IC3: Internet Crime Complaint Center del FBI.

INCIBE: Instituto Nacional de Ciberseguridad de España.

LLM: Gran modelo de lenguaje (Large Language Model).

LSTM: Long Short-Term Memory; tipo de red neuronal recurrente.

M365: Microsoft 365; suite de productividad de Microsoft.

MFA: Autenticación multifactor (Multi-Factor Authentication).

ML: Machine Learning (aprendizaje automático).

MLP: Perceptrón multicapa (Multi-Layer Perceptron).

OAuth: Protocolo estándar de autorización delegada.

OSINT: Inteligencia de fuentes abiertas (Open Source Intelligence).

PhaaS: Phishing-as-a-Service; plataformas que ofrecen kits de phishing por suscripción.

QR: Código de respuesta rápida (Quick Response).

RCNN: Red neuronal convolucional recurrente (Recurrent CNN).

RF: Random Forest; algoritmo de aprendizaje por conjuntos de árboles de decisión.

SIEM: Security Information and Event Management; plataforma de correlación de eventos de seguridad.

SMS: Servicio de mensajes cortos (Short Message Service).

SPF: Sender Policy Framework; mecanismo de autenticación del remitente.

SVM: Máquina de vectores de soporte (Support Vector Machine).

TF-IDF: Term Frequency-Inverse Document Frequency; técnica de vectorización de texto.

UEBA: User and Entity Behavior Analytics; análisis del comportamiento de usuarios y entidades.

URL: Localizador uniforme de recursos (Uniform Resource Locator).

WHOIS: Protocolo de consulta de datos de registro de dominios.

XAI: Inteligencia artificial explicable (Explainable AI).

INDICE

Resumen	2
Abstract.....	3
Agradecimientos.....	4
Índice de figuras y tablas.....	5
Abreviaturas.....	6
Introducción	10
Marco teórico del phishing	11
Concepto y contexto actual	11
Fases de un ataque de phishing	11
Preparación y Recopilación de Información (Reconocimiento).....	12
Creación y Lanzamiento del Cebo (Setup y Ataque)	13
Interacción y Explotación (Colección Inicial)	14
Adquisición y Monetización de los Datos	14
Comparación crítica de modelos de fases.....	15
Puntos de intervención y controles	16
Tipos, clasificación y evolución de los ataques de phishing	16
Introducción a la clasificación	16
Clasificación según el vector de entrega	16
Clasificación según el grado de personalización y objetivo	17
Clasificación según la técnica de explotación	17
Evolución histórica resumida	18
Impacto actual (2025)	18
La ingeniería social como fundamento psicológico y sociotécnico del phishing	18
Principios psicológicos fundamentales explotados	19
Sesgos cognitivos específicos explotados	19
El impacto de la inteligencia artificial generativa en la ingeniería social (2023-2025). ..	19
Consecuencias organizativas y humanas	20
Implicaciones para la detección.....	20
Estado del arte en técnicas de detección y prevención de phishing basadas en Machine Learning y Deep Learning (2018-2025)	21
Evolución de los enfoques de detección.....	21
Técnicas clásicas de Machine Learning (representativas 2019-2022).....	21
Técnicas de Deep Learning y Transformers (2023-2025)	22
Enfoques híbridos y basados en comportamiento (UEBA).....	23
Limitaciones actuales de los enfoques.....	24
Conclusiones del estado del arte y hueco de investigación	24

Metodología y planificación	24
Enfoque metodológico	25
Planificación temporal del proyecto.....	25
Herramientas y tecnologías	26
Validación y control de calidad	26
Aspectos éticos y de protección de datos	27
Diseño e implementación de la aplicación web	28
Alcance del sistema	28
Arquitectura general	28
Flujo de análisis	29
Sistema heurístico	29
Clasificador neuronal	29
Interfaz de usuario	30
Diseño modular.....	30
Limitaciones	31
Pruebas, resultados y discusión	32
Pruebas unitarias y de integración	32
Rendimiento y eficiencia de arranque	32
Dataset de ejemplo.....	32
Resultados del clasificador neuronal.....	32
Comportamiento del sistema heurístico	32
Prueba funcional de la aplicación web	33
Discusión	33
Conclusiones	33
Trabajo futuro	34
Glosario	35
Referencias.....	36
Anexos	37
Anexo A: Estructura del repositorio.....	37
Anexo B: Fragmentos de código fuente	38
B.1. Fachada del análisis heurístico (heurísticas.py).....	38
B.2. Servicio común de análisis (analysis_service.py)	38
B.3. Núcleo del clasificador neuronal (modelo_neural.py).....	39
Anexo C: Guía resumida de instalación y uso	40
C.1. Uso del prototipo	40
Anexo D: Lista de verificación de la validación	40

Introducción

La ciberseguridad se enfrenta a una amenaza persistente en el entorno digital: el phishing. La evolución de las campañas y el uso de inteligencia artificial generativa dificultan la detección basada únicamente en filtros y listas negras. Este escenario justifica estudiar aproximaciones que incorporen el contexto, el comportamiento y la intención del mensaje.

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y evaluar un prototipo de detección de phishing en correos electrónicos que combine un enfoque heurístico explicable con un clasificador automático basado en aprendizaje supervisado.

Para alcanzar este objetivo general se plantean los siguientes objetivos específicos:

- Analizar el fenómeno del phishing, sus fases, tipologías y evolución histórica.
- Estudiar los fundamentos psicológicos de la ingeniería social explotados por este tipo de ataques.
- Revisar el estado del arte en técnicas de detección basadas en machine learning y deep learning.
- Diseñar e implementar un sistema heurístico de señales explicables y un clasificador neuronal TF-IDF + MLP.
- Integrar la aplicación web con la API de Gmail y proporcionar alertas opcionales mediante Telegram.
- Evaluar el comportamiento del sistema mediante pruebas unitarias, de integración y funcionales.

La metodología seguida ha combinado una revisión bibliográfica de fuentes académicas y de industria con un desarrollo iterativo del prototipo. El sistema se ha implementado en Python como una aplicación web modular con un núcleo local compartido: la interfaz Streamlit invoca los servicios de análisis directamente y, opcionalmente, el mismo caso de uso se expone mediante una API HTTP local y una extensión de Gmail. La validación se ha realizado mediante pruebas automatizadas, pruebas funcionales de los flujos principales y evaluación del clasificador sobre datos sintéticos controlados en español e inglés.

La memoria comienza con una introducción y un marco teórico dedicado al fenómeno del phishing, sus fases, tipologías y fundamentos psicológicos. A continuación, se revisa el estado del arte, se explica la metodología de trabajo y se describe el diseño de la aplicación web. Finalmente, se presentan las pruebas y resultados, las conclusiones, el glosario, las referencias y los anexos técnicos.

Marco teórico del phishing

Concepto y contexto actual

El phishing se ha consolidado como una de las ciberamenazas más persistentes y dañinas del panorama actual de la ciberseguridad. Según el Anti-Phishing Working Group (APWG), en el segundo trimestre de 2025 se detectaron 1 130 393 ataques de phishing únicos, el mayor volumen trimestral desde 2023 y parte de una tendencia creciente que no muestra signos de desaceleración (Anti-Phishing Working Group, 2025).

El 2025 Data Breach Investigations Report de Verizon sitúa la participación humana en aproximadamente el 60 % de las brechas analizadas e identifica el phishing entre los vectores iniciales observados. Los informes del FBI muestran, además, que el fraude de correo corporativo puede ocasionar pérdidas económicas relevantes, aunque el impacto varía entre incidentes (Verizon, 2025; Federal Bureau of Investigation, 2024).

El phishing ha evolucionado desde campañas masivas hacia ataques dirigidos y mensajes asistidos por inteligencia artificial generativa. Estas herramientas pueden mejorar la fluidez y la adaptación contextual del texto, lo que reduce el valor de algunos indicadores lingüísticos tradicionales sin volverlos inútiles (Heiding et al., 2024).

En España, el Instituto Nacional de Ciberseguridad (INCIBE) gestionó 97.348 incidentes de ciberseguridad durante 2024, un 16,6 % más que en 2023. Dentro del fraude en línea, el phishing fue la tipología más frecuente, con 21.571 casos registrados (Instituto Nacional de Ciberseguridad, 2025).

Este crecimiento, combinado con la limitada efectividad de las soluciones tradicionales frente a ataques basados en inteligencia artificial, justifica la necesidad de desarrollar nuevas aproximaciones, objetivo central del presente Trabajo Fin de Grado.

Fases de un ataque de phishing

El phishing es un tipo de ataque cibernético que explota la confianza y el comportamiento humano para obtener información sensible, como credenciales de acceso o datos financieros. A diferencia de exploits técnicos directos, el phishing sigue un ciclo estructurado que puede desglosarse en fases secuenciales. Esta segmentación permite no solo entender el mecanismo del ataque, sino también identificar puntos de intervención para su detección y prevención.

Una de las anatomías más completas y actualizadas propone cuatro fases principales en el proceso de phishing: preparación y recopilación de información, creación y lanzamiento del cebo, interacción y explotación, y adquisición y monetización de los datos (Alkhalil et al., 2021). Este modelo integra perspectivas previas, como la "cadena de ataque" (kill chain) de tres fases —vector de amenaza, entrega y explotación— propuesta en análisis de seguridad email (GreatHorn, 2021; Hoxhunt, 2024). En esta memoria se adopta este enfoque, complementándolo con el marco MITRE ATT&CK (MITRE, 2025) y datos empíricos del Verizon DBIR 2025, que analiza 22.000 incidentes y 12.195 brechas confirmadas. A continuación, se detalla cada fase, con énfasis en sus componentes clave.

Figura 2.1: Diagrama de flujo swimlane de las fases de un ataque de phishing.



Fuente: Adaptado de Alkhalil et al. (2021) y MITRE ATT&CK (2025). Ilustra la progresión secuencial con énfasis en interacciones humano-técnicas.

Preparación y Recopilación de Información (Reconocimiento)

Esta fase inicial es fundamental para la personalización y efectividad del ataque. El atacante realiza una investigación exhaustiva sobre el objetivo, ya sea un individuo, organización o grupo amplio. Esto incluye la recopilación de datos públicos o filtrados, como correos electrónicos, perfiles en redes sociales, estructuras organizativas o vulnerabilidades técnicas conocidas.

- Actividades clave: Análisis de fuentes abiertas (OSINT, por sus siglas en inglés), como LinkedIn para spear phishing o bases de datos de brechas pasadas (p. ej., Have I Been Pwned). En ataques masivos, se usan bots para escanear listas de emails.
- Ejemplo: En un spear phishing corporativo, el atacante podría mapear la jerarquía de una empresa para targeting de ejecutivos (whaling).
- Duración y riesgos: La fase de reconocimiento puede prolongarse y no siempre deja señales visibles para la organización. El DBIR 2025 identifica el abuso de credenciales como uno de los vectores iniciales más frecuentes, lo que refuerza la necesidad de limitar la exposición pública y vigilar credenciales comprometidas (Verizon, 2025).

Interrumpir esta fase mediante políticas de privacidad y monitoreo de datos expuestos es un primer nivel de defensa.

Creación y Lanzamiento del Cebo (Setup y Ataque)

Una vez identificados los vectores de amenaza, el atacante diseña el "cebo" —el mensaje o vector de entrega— para que parezca legítimo y urgente. Aquí se aprovecha la ingeniería social para evadir filtros de spam y generar confianza.

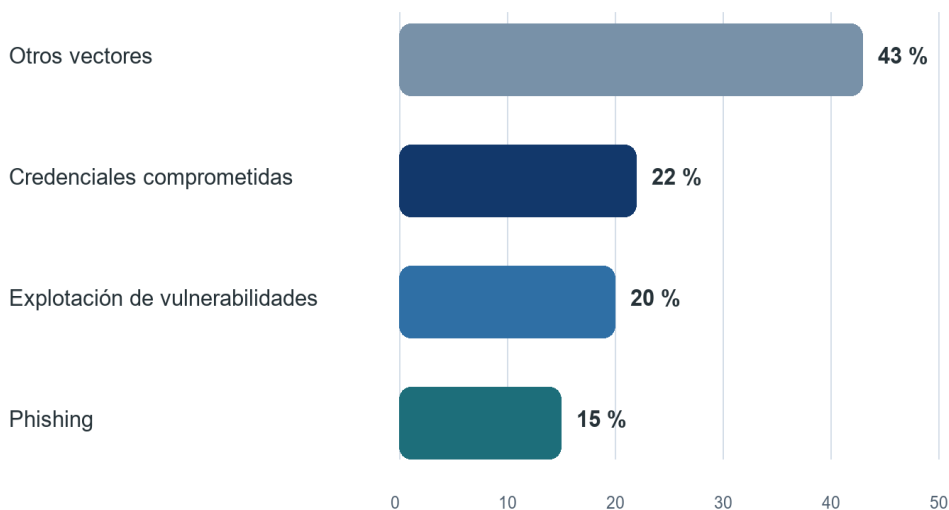
- Actividades clave: Creación de emails falsos, sitios web clonados (usando kits de phishing como Evilginx) o mensajes en otros canales (SMS para smishing, llamadas para vishing). Se incorporan elementos como logos robados, URLs acortadas o adjuntos maliciosos.
- Ejemplo: Un email simulando un banco que advierte de "actividad sospechosa" con un enlace a un sitio falso, o un QR code en un cartel físico que redirige a malware (quishing).
- Técnicas avanzadas: Uso de homógrafos (p. ej., "paypal.com" con caracteres cirílicos) o envenenamiento de DNS para redirigir tráfico (Imperva, 2023).

Esta fase es crítica porque la credibilidad del mensaje y su similitud con comunicaciones reales condicionan la interacción de la víctima. En el DBIR 2025, el phishing representó aproximadamente el 15 % de los vectores iniciales identificados en brechas, por detrás del abuso de credenciales y de la explotación de vulnerabilidades (Verizon, 2025).

Figura 2.2: Vectores iniciales en brechas analizadas por el DBIR 2025 (n=9.891).

Vectores iniciales en brechas

DBIR 2025 · n = 9.891 brechas analizadas



Fuente: Verizon, 2025 Data Breach Investigations Report.

Interacción y Explotación (Colección Inicial)

El objetivo es inducir una acción del usuario, como clicar un enlace, abrir un adjunto o ingresar datos. Aquí ocurre la "mordida" del anzuelo, donde la víctima interactúa con el payload malicioso.

- **Actividades clave:** El usuario puede ser redirigido a un sitio falso para capturar credenciales o inducido a abrir un adjunto malicioso. En ataques de varias etapas, un proxy adversary-in-the-middle (AiTM) puede interceptar tokens de sesión en tiempo real (Kroll, 2025).
- **Ejemplo:** En un ataque de clonación (clone phishing), se duplica un email legítimo previo y se reemplaza un adjunto seguro por uno infectado (Fortinet, 2025).
- **Factores humanos:** La urgencia, el miedo a perder acceso y la apariencia de legitimidad explotan sesgos cognitivos y pueden precipitar una decisión. La formación y la autenticación resistente al phishing reducen el riesgo, pero no eliminan la necesidad de controles técnicos.

Esta fase muestra la intersección entre tecnología y psicología: los controles técnicos deben complementarse con formación, autenticación resistente al phishing y mecanismos de detección. El DBIR 2025 sitúa la participación humana en el 60 % de las brechas analizadas (Verizon, 2025).

Adquisición y Monetización de los Datos

Con los datos en mano, el atacante los procesa y convierte en ganancia. Esta fase cierra el ciclo, pero puede extenderse a ataques secundarios.

- **Actividades clave:** Extracción de credenciales para accesos no autorizados, venta en dark web o uso directo (p. ej., transferencias fraudulentas). En BEC (Business Email Compromise), se roban fondos directamente.

- Ejemplo: En la campaña CorruptQR observada por Kroll, documentos de Office con cabeceras dañadas mostraban un código QR tras su reparación; al escanearlo, la víctima era dirigida a una página de phishing orientada al robo de credenciales corporativas y tokens de sesión (Kroll, 2025).
- Impacto: El coste económico depende del tipo de incidente y no debe atribuirse íntegramente al phishing. Como referencia general, IBM estimó en 4,44 millones de dólares el coste medio mundial de una brecha de datos en 2025, un 9 % menos que el año anterior (IBM, 2025).

Tabla 2.1: Indicadores de contexto del phishing y las brechas (2025).

Indicador	Medida	Valor	Fuente
Actividad observada	Ataques de phishing en Q2 2025	1.130.393	APWG
Acceso inicial	Phishing en brechas analizadas	≈ 15 %	Verizon DBIR
Factor humano	Brechas con participación humana	60 %	Verizon DBIR
Impacto general	Coste medio global por brecha	4,44 M USD	IBM

Comparación crítica de modelos de fases

Tabla 2.2: Comparativa de modelos

Autor/Año	Nº de fases	Nombres de las fases	Ventajas del modelo
GreatHorn (2023)	3	Vector → Delivery → Exploitation	Simplicidad operativa
Alkhalil et al. (2021)	4	Research → Creation → Interaction → Execution	Completo y académico
APWG (2024)	7	Planning → Research → Attack development → Attack → Collection → Fraud → Post-fraud	Detallado extremo
Modelo adoptado	4 + MITRE	Reconocimiento → Distribución → Interacción → Monetización + tácticas MITRE	Equilibrio rigor-calidad

Puntos de intervención y controles

Tabla 2.3: Controles por fase

Fase	Control preventivo	Control detectivo	Herramientas
Reconocimiento	Monitoreo OSINT	Alertas leaks	Have I Been Pwned
Distribución	Filtros email (SPF/DKIM/DMARC)	Análisis QR	Mimecast
Interacción	MFA resistente AiTM	Monitoreo clics	Microsoft Defender
Monetización	Detección anomalías	Análisis exfiltración	SIEM (Splunk, QRadar)

Este modelo de cuatro fases no es rígido; en ataques sofisticados, se solapan o iteran. Entenderlas es esencial para prototipos de detección, como los basados en ML que analizan patrones en la fase 2 (Alkhalil et al., 2021). Estas fases ilustran cómo el phishing ha evolucionado de ataques rudimentarios a campañas orquestadas, sentando las bases para discutir sus tipos y evolución histórica en las siguientes secciones.

Tipos, clasificación y evolución de los ataques de phishing

Introducción a la clasificación

Aunque el objetivo final del phishing es siempre el mismo —obtener información sensible o acceso no autorizado explotando la confianza humana—, la forma de llevarlo a cabo ha diversificado enormemente en los últimos años. En la actualidad, los ataques se clasifican principalmente según tres ejes: el canal o vector empleado, el grado de personalización del ataque y la técnica de explotación utilizada. Esta triple clasificación permite comprender mejor la evolución del fenómeno y anticipar las tendencias futuras.

Clasificación según el vector de entrega

El correo electrónico continúa siendo un canal central del phishing, aunque las campañas combinan cada vez más mensajes, páginas falsas, códigos QR y fraude de correo corporativo. En el segundo trimestre de 2025, APWG observó 1.130.393 ataques de phishing y documentó el uso de códigos QR contra 1.642 marcas (Anti-Phishing Working Group, 2025).

Los mensajes SMS (smishing), las llamadas telefónicas (vishing), las redes sociales y los códigos QR amplían los canales de entrega. Kroll observó en 2024 campañas que empleaban mensajes en redes sociales y documentos reparables que revelaban códigos QR, mientras que APWG dedicó en 2025 un seguimiento específico a los ataques mediante QR (Kroll, 2025; Anti-Phishing Working Group, 2025).

Otros vectores menos frecuentes pero en ascenso son el phishing a través de redes sociales (mensajes privados en LinkedIn, WhatsApp Business o Telegram), el envenenamiento de resultados de búsqueda (SEO poisoning) y el pharming o manipulación de resolución DNS.

Clasificación según el grado de personalización y objetivo

En función del nivel de targeting distinguimos varios tipos claramente diferenciados:

- Phishing masivo o genérico: campañas de gran volumen que reutilizan mensajes similares. Su coste reducido permite mantenerlas incluso cuando solo una fracción pequeña de los destinatarios interactúa.
- Spear-phishing: ataque dirigido contra una persona o un grupo reducido. Requiere investigación previa (OSINT) y mensajes adaptados al contexto, por lo que suele resultar más persuasivo que una campaña genérica.
- Whaling o CEO-fraud: variante del spear-phishing dirigida a altos ejecutivos o perfiles financieros. Puede producir pérdidas elevadas cuando deriva en fraude de correo corporativo, aunque el impacto varía ampliamente entre incidentes (Federal Bureau of Investigation, 2024).
- Clone phishing: se toma un correo legítimo previamente recibido por la víctima y se modifica un enlace o adjunto. Al conservar elementos de una conversación real, puede aumentar la confianza inicial.
- Adversary-in-the-Middle (AiTM): técnica que utiliza proxies inversos para interceptar la autenticación en tiempo real y capturar tokens de sesión incluso cuando la víctima emplea determinados métodos de autenticación multifactor. Constituye un riesgo relevante para servicios corporativos en la nube (Kroll, 2025).

Clasificación según la técnica de explotación

El tercer eje clasifica los ataques según la técnica de explotación, con independencia del canal de entrega o del nivel de personalización. Entre las tendencias recientes destacan el uso abusivo de inteligencia artificial generativa, la oferta de kits como servicio y las técnicas orientadas al robo de tokens:

1. Inteligencia artificial generativa: los modelos de lenguaje pueden ayudar a redactar mensajes fluidos, adaptar el tono al contexto y producir contenido web. Su resultado depende de la información y de las instrucciones disponibles, y no elimina otros indicios técnicos del ataque.
2. Phishing-as-a-Service (PhaaS): plataformas que empaquetan plantillas, alojamiento y mecanismos de evasión, reduciendo la barrera de entrada para campañas sofisticadas. Kroll relacionó CorruptQR con la plataforma ONNX y observó otros kits dirigidos a cuentas Microsoft 365 (Kroll, 2025).
3. Bypass de MFA mediante robo de tokens: los proxies adversary-in-the-middle pueden capturar cookies o tokens de sesión después de una autenticación válida. Por ello se recomiendan métodos resistentes al phishing, como FIDO, junto con controles de acceso condicional (Kroll, 2025).

Evolución histórica resumida

El phishing ha pasado por varias etapas claramente diferenciadas:

- 1995-2003: orígenes en AOL y primeros ataques masivos a PayPal.
- 2004-2010: aparición de kits (Rock Phish, Avalanche) y profesionalización.
- 2011-2016: auge del spear-phishing y del ransomware entregado vía phishing.
- 2016-2020: explosión del Business Email Compromise (BEC).
- 2020-2022: pandemia y boom de smishing/vishing.
- 2022-2025: expansión de la IA generativa, AiTM, quishing y PhaaS, con una mayor combinación de canales, automatización y servicios criminales.

En tres décadas, el phishing ha pasado de campañas rudimentarias a operaciones criminales organizadas que combinan automatización, ingeniería social y servicios comercializados en mercados clandestinos.

Impacto actual (2025)

El volumen sigue siendo elevado: APWG observó 1.130.393 ataques de phishing en el segundo trimestre de 2025. En España, INCIBE gestionó 97.348 incidentes de ciberseguridad durante 2024, incluidos 21.571 casos de phishing. Como indicador económico general, no específico de phishing, IBM situó el coste medio mundial de una brecha en 4,44 millones de dólares en 2025 (Anti-Phishing Working Group, 2025; Instituto Nacional de Ciberseguridad, 2025; IBM, 2025).

Este panorama justifica complementar los filtros de correo y las listas negras con sistemas capaces de analizar el contexto, el comportamiento y la intención del mensaje, objetivo principal del presente trabajo.

La ingeniería social como fundamento psicológico y sociotécnico del phishing

Aunque el phishing se manifiesta a través de medios técnicos —correo electrónico, SMS, sitios web falsos o códigos QR—, su eficacia depende en gran medida de la explotación de factores cognitivos, emocionales y sociales. Por ello, la ingeniería social constituye un componente central del fenómeno (Hadnagy, 2018).

Principios psicológicos fundamentales explotados

Los atacantes diseñan sus campañas apoyándose en principios psicológicos bien documentados, muchos de ellos identificados originalmente por Robert Cialdini (1984) y posteriormente validados en el ámbito de la ciberseguridad:

Tabla 2.4: Aplicación de los principios de Cialdini en campañas de phishing.

Principio	Aplicación habitual en phishing	Riesgo para la víctima
Autoridad	Suplantación de bancos, administraciones, dirección o proveedores	Reduce el cuestionamiento de la solicitud
Urgencia / escasez	Plazos breves, bloqueo de cuentas o falsas oportunidades	Favorece decisiones precipitadas
Reciprocidad	Regalos, descuentos o supuestas actualizaciones gratuitas	Normaliza la primera interacción
Compromiso y coherencia	Secuencias que comienzan con una acción pequeña	Aumenta la probabilidad de continuar
Prueba social	Avisos sobre acciones atribuidas a otros usuarios	Hace que la petición parezca habitual
Simpatía	Identidades conocidas, tono cercano o afinidad profesional	Incrementa la confianza inicial

Sesgos cognitivos específicos explotados

Más allá de los principios de influencia, los atacantes aprovechan sesgos cognitivos automáticos (Kahneman, 2011; Tversky & Kahneman, 1974) que operan a nivel Sistema 1 (rápido, intuitivo):

- Sesgo de anclaje: el usuario ancla su percepción en el remitente aparentemente legítimo.
- Efecto de mera exposición: cuanto más parecido sea el dominio o el diseño al original, mayor confianza genera.
- Aversión a la pérdida (loss aversion): el miedo a perder el acceso a una cuenta o sufrir una penalización puede pesar más que una ganancia equivalente y precipitar la respuesta.
- Fatiga de decisión y sobrecarga cognitiva: en entornos laborales con alto volumen de correo, la atención selectiva colapsa (GreatHorn, 2021).

La combinación de urgencia, autoridad y prueba social puede elevar la persuasión del mensaje. Este efecto depende del contexto, de la experiencia previa y de las diferencias individuales, por lo que no debe expresarse como una tasa universal de éxito (Heiding et al., 2024).

El impacto de la inteligencia artificial generativa en la ingeniería social (2023-2025)

La disponibilidad de modelos de lenguaje grandes (LLM) ha introducido cambios cualitativos en la preparación de mensajes de ingeniería social:

- Reducción de indicadores lingüísticos tradicionales: los modelos generativos pueden producir mensajes fluidos y disminuir errores gramaticales o construcciones extrañas que antes facilitaban la detección (Heiding et al., 2024).
- Personalización contextual: los modelos de lenguaje pueden adaptar el tono, incorporar información pública y reproducir terminología propia de un sector, aunque la calidad depende de los datos y de las instrucciones disponibles.
- Generación dinámica de narrativas: las herramientas generativas pueden producir y adaptar textos con rapidez. Su uso abusivo reduce el esfuerzo de personalización, aunque no garantiza que el mensaje resulte creíble ni eficaz.

Los experimentos de Heiding et al. (2024) muestran que los modelos de lenguaje pueden producir correos de phishing competitivos frente a mensajes redactados por personas y que la incorporación de principios de persuasión modifica su eficacia. Los resultados corresponden a un diseño experimental concreto y no constituyen una tasa universal de éxito.

Consecuencias organizativas y humanas

El Verizon DBIR 2025 sitúa la participación humana en el 60 % de las brechas analizadas. En España, el 33 % de los usuarios que contactaron con la línea 017 indicó haber recibido algún intento de phishing, vishing o smishing durante 2024 (Verizon, 2025; Instituto Nacional de Ciberseguridad, 2025).

Este factor humano no es estático: la fatiga por alerta, la sobrecarga de correo y la normalización del trabajo remoto reducen la atención sostenida. Los informes de industria señalan además que determinados perfiles, como finanzas, recursos humanos o dirección, reciben un volumen desproporcionado de ataques dirigidos (Proofpoint, 2025a).

Implicaciones para la detección

La dimensión psicológica del phishing implica que una solución eficaz debe combinar el análisis técnico con señales del contexto humano. Los sistemas basados únicamente en características estáticas de URL o contenido pueden degradarse cuando cambian las campañas o se emplea texto generado (Alghenaim et al., 2025; Heiding et al., 2024).

Por tanto, entre las líneas de evolución de la detección se encuentran los modelos que incorporan:

- Análisis semántico profundo del texto (transformers, LLMs).
- Modelado del comportamiento del usuario (UEBA).
- Detección de intención y manipulación psicológica mediante NLP avanzado.

Este será precisamente el fundamento teórico sobre el que se construya el estado del arte y el prototipo propuesto en las siguientes secciones.

Estado del arte en técnicas de detección y prevención de phishing basadas en Machine Learning y Deep Learning (2018-2025)

La detección de phishing ha evolucionado desde reglas y listas de bloqueo hacia modelos de machine learning y deep learning capaces de combinar rasgos de URL, contenido, cabeceras y contexto. Las cifras publicadas no son directamente comparables porque varían el corpus, el equilibrio de clases, la partición de datos y la definición de phishing; por ello, esta revisión prioriza las aportaciones y limitaciones de cada enfoque sobre una clasificación basada únicamente en accuracy (Thakur et al., 2023; Alghenaim et al., 2025).

La sección revisa trabajos académicos e informes técnicos entre 2018 y 2025. Se organiza en enfoques clásicos, deep learning, transformers, métodos híbridos y limitaciones, y utiliza las revisiones citadas para contextualizar estudios representativos sin presentar sus métricas como si procedieran de un único benchmark.

Evolución de los enfoques de detección

La progresión se divide en tres fases principales, alineadas con la madurez tecnológica:

1. 2019-2021: Enfoques clásicos basados en características manuales. Random Forest, SVM y otros clasificadores trabajan con URL, HTML, cabeceras y variables léxicas. Son rápidos y relativamente explicables, pero dependen de rasgos que pueden envejecer cuando cambian las tácticas.
2. 2022-2023: Consolidación del deep learning. CNN, LSTM y modelos preentrenados reducen parte de la ingeniería manual y aprenden patrones en URLs o texto. Su comparación sigue limitada por corpus distintos y por evaluaciones realizadas en entornos controlados.
3. 2024-2025: Transformers y modelos de lenguaje. Los trabajos recientes estudian clasificación zero-shot, ajuste fino y explicaciones generadas por LLM. El potencial es alto, pero la precisión, la fidelidad de las explicaciones, el coste y la privacidad no mejoran siempre de forma simultánea (Koide et al., 2024; Kuikel et al., 2025).

Esta evolución refleja el paso de reglas fijas a modelos que aprenden representaciones más ricas. No elimina la utilidad de los enfoques clásicos: en despliegues locales, la velocidad, la trazabilidad y el coste siguen siendo criterios relevantes.

Técnicas clásicas de Machine Learning (representativas 2019-2022)

Estos métodos priorizan eficiencia computacional y explicabilidad, pero requieren feature engineering manual.

Tabla 3.1: Comparativa de técnicas clásicas de machine learning.

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2018	Hutchinson et al.	Random Forest	Sitios web	Muestra la utilidad de rasgos de URL y página	Depende de variables diseñadas manualmente

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2020	Yerima y Alzaylaee	Red convolucional	URLs y páginas	Aprende representaciones con menos ingeniería manual	No cubre por sí sola la semántica completa del correo
2020	AlEroud y Karabatis	Red adversarial	Evasión de detectores	Estudia robustez frente a ejemplos generados	La generalización fuera del laboratorio es difícil
2022	Sánchez-Paniagua et al.	Dataset multipropósito	Sitios web	Facilita comparaciones reproducibles	No representa todos los canales de phishing

(Fuentes: Hutchinson et al., 2018; Yerima & Alzaylaee, 2020; AlEroud & Karabatis, 2020; Sánchez-Paniagua et al., 2022).

Los enfoques clásicos extraen características de URLs, páginas, cabeceras o texto y las proporcionan a clasificadores supervisados. Su rendimiento puede ser elevado dentro del corpus utilizado, pero cae cuando la distribución cambia o el atacante modifica los rasgos que el modelo espera.

Yerima y Alzaylaee (2020) aplicaron redes convolucionales a la detección de phishing, mientras que AlEroud y Karabatis (2020) estudiaron ataques adversariales contra detectores. En conjunto, estos trabajos desplazan el foco desde la precisión estática hacia la capacidad de aprender representaciones y resistir evasiones.

Sánchez-Paniagua et al. (2022) publicaron un dataset multipropósito y características de tecnologías web para facilitar comparaciones reproducibles. La limitación común de esta generación es que un buen resultado sobre páginas o URLs no demuestra por sí solo el comportamiento ante correos nuevos, idiomas distintos o campañas generadas por IA.

Técnicas de Deep Learning y Transformers (2023-2025)

Los avances recientes enfatizan multimodalidad y zero-shot, con énfasis en LLMs para detección semántica.

Tabla 3.2: Comparativa de técnicas de deep learning y transformers.

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2023	Thakur et al.	Revisión sistemática	Correo	Sintetiza arquitecturas de deep learning	Los datasets y métricas son heterogéneos
2023	Wang et al.	Modelo preentrenado	URLs	Explora transferencia y detección a gran escala	Se centra en la URL, no en el mensaje completo
2024	Altwaijry et al.	Comparativa DL	Correo	Compara varias arquitecturas sobre una misma tarea	El resultado depende del corpus evaluado
2024	Koide et al.	LLM zero-shot	Correo	Evalúa detección sin ajuste específico	Coste, privacidad y reproducibilidad

Año	Trabajo	Enfoque	Objeto	Aportación	Limitación
2024	Rojas-Galeano	LLM zero-shot	Spam	Analiza transferencia entre modelos preentrenados	Spam y phishing no son categorías equivalentes
2024	Uddin et al.	Transformer explicable	Correo	Relaciona predicción con términos relevantes	Preprint pendiente de validación más amplia
2025	Kuikel et al.	LLM + explicaciones	Correo	Compara precisión y alineación de explicaciones	Existe un compromiso entre acierto y fidelidad
2025	Alghenaim et al.	Revisión del estado del arte	Detección con IA	Resume tendencias, riesgos y líneas abiertas	No es un clasificador evaluado de forma original

(Fuentes: Thakur et al., 2023; Wang et al., 2023; Altwaijry et al., 2024; Koide et al., 2024; Rojas-Galeano, 2024; Uddin et al., 2024; Kuikel et al., 2025; Alghenaim et al., 2025).

Los trabajos recientes amplían el objeto de análisis, pero difieren en corpus, tarea y protocolo de evaluación. La tabla resume su aportación principal sin mezclar resultados que no proceden del mismo benchmark.

Las revisiones de Thakur et al. (2023) y Alghenaim et al. (2025) describen una transición hacia redes profundas, modelos preentrenados y combinaciones de texto, URL y metadatos. Este avance reduce parte de la ingeniería manual, aunque mantiene problemas de comparabilidad y deriva temporal.

Altwaijry et al. (2024) compararon modelos de deep learning para correo y Koide et al. (2024) exploraron un detector basado en modelos de lenguaje. Ambos trabajos muestran el interés por incorporar contexto semántico, pero no justifican trasladar una cifra de accuracy a otros corpus o entornos.

Rojas-Galeano (2024) estudió clasificación zero-shot de spam, una tarea próxima pero no idéntica al phishing. Uddin et al. (2024) abordaron la explicabilidad con transformers y Kuikel et al. (2025) observaron que una mejor alineación entre predicción y explicación no implica necesariamente mayor precisión. Esta tensión es central para sistemas de apoyo a la decisión.

Enfoques híbridos y basados en comportamiento (UEBA)

Los enfoques híbridos combinan señales estructurales, contenido y comportamiento para aportar contexto que una única fuente no contiene. La integración puede mejorar cobertura, pero aumenta la complejidad de datos, despliegue y evaluación.

- Híbridos semánticos e ingeniería social: combinan representaciones del texto con señales de urgencia, autoridad o solicitud de credenciales. Su principal reto es demostrar que esas señales generalizan fuera del corpus de entrenamiento.
- Multimodal: fusiona texto, URL, imágenes y metadatos. Puede detectar señales complementarias, aunque exige datasets alineados y eleva el coste de entrenamiento e inferencia.

- Zero-shot: utiliza modelos preentrenados sin ajuste específico. Es útil para explorar ataques emergentes, pero introduce variabilidad, coste y posibles transferencias de datos a servicios externos.
- Prevención proactiva: UEBA analiza cambios de comportamiento y anomalías de acceso. Complementa la inspección del mensaje, aunque requiere telemetría organizativa que queda fuera del alcance de este prototipo.

La literatura reciente presenta los métodos híbridos como una línea prometedora, pero subraya la necesidad de evaluar cada componente y evitar que la complejidad oculte qué señal sostiene la decisión (Alghenaim et al., 2025; Popescul y Radu, 2025).

Limitaciones actuales de los enfoques

Aunque distintos trabajos publican resultados elevados en sus propios corpus, persisten desafíos que impiden compararlos directamente:

1. Datasets desfasados: muchos corpus no reflejan campañas recientes, IA generativa, quishing ni deriva de dominios. Un resultado interno elevado puede degradarse ante ataques nuevos.
2. Coste y latencia: los modelos grandes exigen más memoria, cómputo o llamadas a servicios externos; los modelos locales ligeros ofrecen mayor control y reproducibilidad.
3. Multilingüismo y sesgo: el predominio de corpus en inglés dificulta trasladar resultados al español y a contextos culturales diferentes.
4. Robustez adversarial: homógrafos, ofuscación, cambios de plantilla y texto generado pueden eludir patrones aprendidos. Pocos trabajos evalúan de forma comparable estas evasiones.
5. Explicabilidad: una explicación generada no garantiza fidelidad al proceso de decisión. Debe distinguirse entre texto persuasivo y evidencia verificable del modelo.

La revisión bibliométrica de Popescul y Radu (2025) confirma el crecimiento de ML, deep learning y modelos híbridos, y señala como líneas abiertas la adaptación a nuevos vectores, la robustez y la integración organizativa.

Conclusiones del estado del arte y hueco de investigación

La literatura reciente sitúa los transformers y los LLM entre las líneas activas de investigación, pero advierte que los resultados de entornos controlados no son directamente comparables. Persisten tres huecos relevantes:

1. Falta de fine-tuning open-source para español/multilingüe
2. Escasa integración de detección psicológica (Cialdini) en LLMs
3. Evaluación limitada contra ataques GenAI en tiempo real.

Metodología y planificación

Este capítulo describe el enfoque metodológico seguido para el desarrollo del prototipo, así como la planificación temporal del proyecto y las herramientas empleadas. El trabajo se ha desarrollado a lo largo del curso académico 2025-2026, distribuyendo las actividades de análisis y documentación en el primer cuatrimestre y las de implementación y validación en el segundo.

Enfoque metodológico

El proyecto se ha desarrollado siguiendo una metodología iterativa e incremental, combinada con una revisión bibliográfica continua. Cada iteración añade una capacidad nueva al sistema y se cierra con la ejecución de la suite de pruebas, de modo que el prototipo permanece en un estado funcional durante todo el proceso.

La fase inicial, desarrollada entre octubre y diciembre de 2025, se dedicó al estudio del contexto del problema y a la revisión de la literatura académica (IEEE, arXiv y Springer) y de los informes de industria citados en el estado del arte. Esta etapa permitió identificar las técnicas de detección más relevantes, delimitar el hueco de investigación y definir los objetivos específicos y el alcance del sistema.

Una vez consolidado el marco teórico, se adoptó un enfoque de desarrollo en espiral adaptado al tamaño del trabajo, llevado a cabo entre febrero y junio de 2026. Primero se construyó un núcleo heurístico explicable, después se añadió el clasificador neuronal TF-IDF + MLP y, a continuación, se desarrolló la aplicación Streamlit con sus vistas de detección, configuración, entrenamiento y monitorización. La integración con Gmail y Telegram reutiliza los mismos módulos locales de análisis. Cada avance se cerró con la ejecución de la suite de pruebas, manteniendo el prototipo en un estado funcional durante todo el proceso.

La gestión del proyecto se apoyó en el control de versiones Git y en la plataforma GitHub como repositorio remoto. El desarrollo se organizó por funcionalidades y cada hito se validó mediante pruebas automatizadas antes de integrarse en la versión de trabajo, con especial atención a las funciones críticas de detección, normalización y persistencia de modelos.

Planificación temporal del proyecto

La planificación se distribuyó en siete fases repartidas a lo largo del curso académico. La tabla siguiente resume la planificación temporal, las actividades principales de cada fase y el entregable asociado. Las fases de análisis y documentación se concentraron en el primer cuatrimestre, mientras que la implementación del prototipo y su validación se desarrollaron en el segundo.

Tabla 4.1: Planificación temporal del proyecto.

Fase	Periodo	Actividades principales	Entregable
Fase 1: Análisis del contexto y estado del arte	Octubre - diciembre 2025	Revisión bibliográfica, análisis del phishing, sus tipologías, la ingeniería social y las técnicas de detección.	Marco teórico y estado del arte
Fase 2: Diseño funcional y modular	Enero - febrero 2026	Definición del alcance de la aplicación web, separación de responsabilidades y selección de tecnologías.	Especificación funcional y diseño modular
Fase 3: Motor heurístico	Febrero - marzo 2026	Implementación del analizador, señales de cabeceras, contenido, URLs y HTML, y cálculo del riesgo.	Núcleo heurístico con pruebas
Fase 4: Clasificador neuronal	Marzo - abril 2026	Pipeline TF-IDF + MLP, entrenamiento en español e inglés y comparación de configuraciones.	Modelos neuronales entrenables
Fase 5: Aplicación web	Abril - mayo 2026	Desarrollo de las vistas Streamlit de inicio, configuración, detección, monitorización y entrenamiento.	Aplicación web funcional
Fase 6: Integraciones y validación	Mayo - junio 2026	Integración OAuth con Gmail, alertas de Telegram, pruebas automatizadas y evaluación funcional.	Prototipo integrado y resultados

Fase	Periodo	Actividades principales	Entregable
Fase 7: Documentación final	Junio 2026	Validación final, redacción de la memoria y revisión de la documentación.	Memoria del TFG

La carga de trabajo del segundo cuatrimestre se concentró en el motor heurístico, el clasificador neuronal y la interfaz web, por ser los componentes que reúnen la lógica central del prototipo. Las integraciones con Gmail y Telegram se incorporaron posteriormente reutilizando el mismo servicio de análisis; además, se añadió una API HTTP local y un adaptador para la extensión de Gmail sin duplicar las reglas del dominio.

Herramientas y tecnologías

La tabla siguiente recoge las principales herramientas y tecnologías utilizadas durante el desarrollo, junto con el propósito de cada una en el proyecto.

Tabla 4.2: Herramientas y tecnologías utilizadas.

Herramienta / tecnología	Propósito en el proyecto
Python 3	Lenguaje de programación principal del prototipo.
Streamlit	Interfaz web con vistas de inicio, configuración, detección, monitorización y entrenamiento.
BeautifulSoup4	Procesamiento del HTML incluido en los correos y extracción de señales estructurales.
NumPy	Operaciones numéricas utilizadas durante el entrenamiento y la evaluación.
scikit-learn	Vectorizador TF-IDF, clasificador MLPClassifier y métricas de evaluación.
joblib	Persistencia en disco de los modelos neuronales entrenados.
langdetect	Selección del modelo en español o inglés según el contenido del correo.
Gmail API (OAuth)	Importación autorizada de correos desde la bandeja del usuario.
Bot API de Telegram	Envío opcional de alertas desde la monitorización de Gmail.
unittest	Pruebas unitarias y de integración de los componentes del prototipo.
Git y GitHub	Control de versiones, trazabilidad y respaldo del desarrollo.

Validación y control de calidad

La validación del sistema se ha realizado en tres niveles complementarios. En primer lugar, la versión final ejecuta 47 pruebas unitarias y de integración con unittest. Estas pruebas cubren el parseo de archivos .eml, las reglas heurísticas, el servicio común de análisis, la configuración, la integración con Gmail, la monitorización, el clasificador neuronal, la persistencia de modelos y las notificaciones por Telegram.

En segundo lugar, la aplicación permite evaluar el clasificador neuronal con un CSV de prueba independiente, separado del conjunto de entrenamiento. La interfaz calcula exactitud, precisión, recall, F1, accuracy balanceada y la matriz de confusión; no se fija una partición automática 70/30 ni se presentan resultados estadísticos sin indicar el conjunto utilizado. En tercer lugar, se comprobaron los flujos funcionales de la aplicación web: análisis de texto pegado y archivos .eml, selección del modelo por idioma, importación mediante

Gmail y navegación entre las vistas de detección y entrenamiento. Los resultados reproducibles se presentan en la sección de pruebas y discusión.

Aspectos éticos y de protección de datos

El tratamiento de los correos electrónicos analizados por el prototipo se plantea conforme al Reglamento General de Protección de Datos (RGPD, Reglamento (UE) 2016/679). El sistema minimiza la información procesada: solo extrae los metadatos y las señales necesarias para la clasificación (cabeceras, remitente, asunto, enlaces y estructura HTML), sin almacenar de forma persistente el contenido íntegro de los mensajes, las credenciales de acceso ni los datos personales de los remitentes.

El acceso a la API de Gmail se realiza mediante OAuth, delegando en Google la autenticación del usuario y evitando el manejo de contraseñas en la aplicación. El prototipo está diseñado para entornos controlados y académicos, con datos de demostración o con el consentimiento explícito del usuario. Los tokens y credenciales se almacenan localmente y quedan excluidos del repositorio. Antes de publicar la aplicación para varios usuarios sería necesario incorporar autenticación, gestión segura de secretos y una política explícita de conservación de datos.

Diseño e implementación de la aplicación web

Tras el análisis teórico de las técnicas de phishing y de los principales enfoques de detección existentes, se ha desarrollado un prototipo funcional orientado al análisis de correos electrónicos sospechosos. El objetivo del sistema no es sustituir a una solución empresarial completa de seguridad, sino demostrar cómo puede combinarse un enfoque heurístico explicable con un clasificador automático basado en aprendizaje supervisado.

El prototipo se centra en el análisis de mensajes de correo electrónico desde una aplicación web y varias entradas locales. El origen puede ser texto pegado, un archivo .eml, un mensaje importado mediante Gmail, el correo visible en la extensión de navegador o un JSON enviado a la API local. A partir de la entrada, el sistema extrae cabeceras, remitente, asunto, cuerpo en texto plano, contenido HTML, enlaces, anclas y adjuntos, y genera una evaluación de riesgo mediante reglas heurísticas, modelo neuronal o una combinación ponderada.

Alcance del sistema

El alcance del prototipo se ha delimitado al análisis estático del correo electrónico y de las URLs contenidas en él. El sistema no navega activamente hacia las páginas enlazadas, no descarga contenido externo ni consulta servicios de reputación en tiempo real. La API, la extensión, Streamlit y los procesos de monitorización se mantienen en el entorno local y no convierten el proyecto en un servicio remoto multiusuario.

No obstante, la arquitectura queda preparada para ampliaciones futuras. Podrían añadirse consultas a listas negras públicas, reputación de dominios, validación de certificados TLS, análisis dinámico de páginas o integración con sistemas corporativos de alerta. El prototipo no se plantea, por tanto, como un producto cerrado, sino como una base funcional y extensible.

Arquitectura general

La solución adopta una arquitectura modular con un núcleo local compartido. Streamlit proporciona la capa de presentación y organiza las vistas de inicio, configuración, detección, monitorización y entrenamiento; estas vistas pueden invocar directamente los servicios del paquete sistema_phishing. De forma opcional, el mismo caso de uso se expone mediante un servidor HTTP local y una extensión de Gmail, sin enviar el contenido a un servicio externo.

A nivel interno, el paquete sistema_phishing agrupa los componentes del dominio. analysis_service.py coordina los modos heurístico, neuronal y combinado y selecciona el modelo según el idioma del correo. analizador_email.py normaliza texto y archivos .eml; correo.py define la representación común; signal_builder.py construye las señales; scorer.py calcula la puntuación; explanations.py genera explicaciones; y modelo_neural.py encapsula el pipeline TF-IDF + MLP y su persistencia. Las vistas Streamlit consumen estas funciones mediante llamadas locales.

Las reglas heurísticas se han dividido en módulos especializados: header_signals.py para cabeceras y autenticación, content_signals.py para contenido textual y adjuntos, html_signals.py para estructuras HTML sospechosas y url_utils.py para extracción y análisis de URLs y dominios. La detección del idioma está centralizada en idioma.py, de modo que la aplicación y la monitorización seleccionan el modelo neuronal con el mismo criterio. Esta división mejora la mantenibilidad y facilita añadir nuevas reglas sin modificar el núcleo del analizador.

Flujo de análisis

El flujo general comienza con la entrada del correo. Si el usuario sube un archivo .eml, el parser extrae remitente, destinatario, asunto, cuerpo en texto plano, cuerpo HTML, cabeceras, enlaces, anclas y adjuntos. Si el usuario pega el contenido manualmente, se genera una representación simplificada. Cuando la entrada procede de Gmail, la propia aplicación obtiene los mensajes autorizados mediante OAuth y los adapta al mismo formato interno.

A continuación, la entrada se normaliza en una representación interna común (CorreoAnalizado), de modo que las reglas heurísticas no dependen del formato original del mensaje.

Una vez normalizado el correo, el componente SignalBuilder ejecuta las reglas de detección y genera un diccionario de señales booleanas. Cada señal representa un posible indicador de phishing, como la existencia de un Reply-To diferente al From, fallos SPF/DKIM/DMARC, enlaces con dominios sospechosos, formularios HTML, redirecciones mediante JavaScript o presencia de lenguaje urgente. Si el modo de análisis lo requiere, se detecta el idioma del texto y se carga el modelo neuronal correspondiente.

RiskScorer asigna una ponderación a cada señal activa y calcula una puntuación final entre 0 y 100. Si la puntuación supera el umbral configurado, el mensaje se clasifica como phishing probable. ExplanationBuilder transforma las señales en explicaciones comprensibles. El mismo servicio puede producir el resultado desde Streamlit, la API local, la extensión o el monitor; si se supera el umbral, la monitorización puede enviar una alerta por Telegram y registrar el identificador del mensaje para evitar duplicados.

Sistema heurístico

El sistema heurístico constituye la parte más explicable del prototipo. Está formado por un conjunto de reglas diseñadas a partir de patrones habituales en correos de phishing. Estas reglas se agrupan en varias categorías.

En primer lugar, se analizan las cabeceras del correo. El sistema comprueba incoherencias entre From, Reply-To, Return-Path y Received-SPF, así como fallos de autenticación SPF, DKIM o DMARC. También se evalúan anomalías en cabeceras Received, dominios sospechosos en Message-ID y posibles intentos de suplantación mediante el nombre visible del remitente.

En segundo lugar, se analizan los enlaces. El sistema extrae URLs del texto y del HTML, comprueba la presencia de dominios en listas negras locales, detecta acortadores conocidos, identifica dominios con punycode o caracteres Unicode, y revisa parámetros de redirección sospechosos.

En tercer lugar, se estudia el contenido HTML. Se detectan formularios con acciones vacías, relativas o dirigidas a dominios sospechosos, así como elementos como iframe, base href, enlaces javascript: o redirecciones mediante meta refresh.

Por último, se analizan rasgos de ingeniería social presentes en el texto, como saludos genéricos, lenguaje urgente, asuntos sospechosos, solicitudes de credenciales o referencias a documentos adjuntos.

Clasificador neuronal

Además del sistema heurístico, se ha implementado un clasificador neuronal basado en scikit-learn. El modelo utiliza un pipeline formado por un vectorizador TF-IDF y un clasificador

MLPClassifier. El vectorizador transforma el texto del correo en una representación numérica basada en n-gramas (por defecto unigramas y bigramas) con un máximo de características configurable. Los hiperparámetros de la red (capas ocultas, función de activación, regularización, learning rate, early stopping, etc.) están centralizados en la clase HiperparametrosModelo y pueden ajustarse desde el entorno (.env.local) o desde la interfaz de entrenamiento.

El sistema permite entrenar modelos en español e inglés, utilizando stopwords específicas para cada idioma. La aplicación de entrenamiento acepta uno o varios archivos CSV, permite configurar las columnas de texto, asunto, cuerpo y etiqueta, y admite formatos habituales de datasets públicos. Durante la evaluación se muestran la matriz de confusión, accuracy, precisión, recall, F1 y accuracy balanceada. También es posible entrenar hasta tres redes en memoria, compararlas sobre un conjunto de prueba independiente y combinar varios CSV de entrenamiento.

Los modelos entrenados se almacenan en disco mediante joblib, diferenciando entre el modelo en español (modelo_neural_es.joblib) y el modelo en inglés (modelo_neural_en.joblib). Durante el análisis, el sistema detecta el idioma del correo con langdetect y reutiliza un detector cacheado por idioma, no uno fijado para toda la sesión. Si el modelo del idioma esperado no existe, está corrupto o tiene metadatos incompatibles, se entrena un modelo sintético del mismo idioma; nunca se reutiliza silenciosamente el modelo de la lengua opuesta.

Interfaz de usuario

La aplicación principal se desarrolla con Streamlit (src/app.py) y organiza la interfaz en cinco vistas: Inicio, Configuración, Detección, Monitor y Entrenamiento. La vista de detección permite analizar texto pegado, archivos .eml o correos importados desde Gmail y seleccionar el modo heurístico, neuronal o combinado. La vista de configuración gestiona Gmail, Telegram, los parámetros del monitor y los hiperparámetros del modelo. La vista de entrenamiento permite cargar CSV, entrenar hasta tres configuraciones y comparar sus métricas.

El resultado se muestra mediante una puntuación de riesgo, una clasificación global y un panel de detalles. En el análisis heurístico, el usuario puede consultar qué señales se han activado y leer una explicación de cada una. Esta decisión busca mejorar la transparencia del sistema, evitando que el resultado sea una simple etiqueta opaca.

La integración con Gmail se realiza desde la propia aplicación mediante OAuth. La vista de detección permite consultar e importar mensajes, mientras que la vista Monitor ejecuta comprobaciones bajo demanda y muestra el resultado de cada correo. De forma opcional, el proceso monitor_gmail.py reutiliza el mismo servicio local para realizar comprobaciones periódicas y enviar alertas por Telegram, sin introducir una capa de análisis remota.

Diseño modular

Durante el desarrollo se ha realizado una refactorización orientada a mejorar la mantenibilidad y aplicar principios de diseño SOLID. El sistema separa la presentación Streamlit, la coordinación de los casos de uso, la construcción de señales, la generación de explicaciones, el cálculo de riesgo, el procesamiento de datasets y la lógica del modelo neuronal. La interfaz depende de servicios locales estables y no de detalles internos de cada regla o clasificador.

Esta organización permite que cada módulo tenga una responsabilidad principal. Por ejemplo, SignalBuilder no calcula puntuaciones, sino que únicamente construye señales; RiskScorer no conoce cómo se han obtenido las señales, solo calcula el riesgo; y ExplanationBuilder no decide si un correo es phishing, sino que transforma resultados técnicos en texto comprensible.

Esta separación facilita la evolución del prototipo. Pueden añadirse reglas en los módulos de señales, sustituirse el clasificador neuronal o incorporarse nuevas fuentes de entrada sin alterar la navegación de la aplicación. La versión final ejecuta correctamente 47 pruebas automatizadas que protegen los componentes principales y sus integraciones locales.

Limitaciones

El prototipo presenta varias limitaciones. Las listas negras son locales y no se consultan servicios externos de reputación en tiempo real. Tampoco se realiza análisis dinámico de las páginas enlazadas, ni se validan certificados digitales ni se comprueba la reputación histórica de los dominios. Además, Streamlit se utiliza como interfaz académica en un entorno controlado: la versión actual no incorpora gestión multiusuario, control de acceso ni aislamiento de datos para un despliegue público.

El modelo neuronal depende de la calidad y representatividad de los datasets utilizados para el entrenamiento. Aunque la aplicación permite entrenar con CSV externos, su rendimiento real debe evaluarse con conjuntos amplios, recientes y equilibrados. Por último, el prototipo no incorpora transformers ni LLMs, aunque el estado del arte los identifica como una línea prometedora junto con la detección semántica de principios de ingeniería social.

Pruebas, resultados y discusión

Pruebas unitarias y de integración

La versión final ejecuta 47 pruebas unitarias y de integración mediante unittest. Cubren el parseo de .eml, la construcción de resultados combinados, la carga de configuración, la decodificación de mensajes de Gmail, la monitorización, las reglas heurísticas, el clasificador neuronal, la persistencia de modelos, la API HTTP y las notificaciones por Telegram. Todas finalizaron correctamente; los únicos avisos observados corresponden a iteraciones reducidas del MLP en pruebas rápidas y no implican fallos funcionales.

Rendimiento y eficiencia de arranque

Para cuantificar el coste operativo se añadió un benchmark reproducible que mide el arranque, el análisis heurístico, la detección de idioma, la carga de modelos y la inferencia. Cada microbenchmark se ejecutó con cinco repeticiones y se tomó la mediana por operación; las importaciones se midieron en procesos nuevos para evitar que la caché de módulos ocultase el coste de inicio.

El perfilado localizó el principal cuello de botella en las importaciones anticipadas. Importar únicamente las heurísticas requería 1098,5090 ms y, tras diferir la carga de scikit-learn y del subsistema neuronal, pasó a 37,0194 ms: una reducción del 96,6 % y una aceleración de 29,7 veces. El arranque frío de la aplicación Streamlit bajó de 1326,5726 ms a 315,1490 ms, una mejora del 76,2 % y 4,2 veces.

Las rutas de inferencia permanecieron estables: el análisis heurístico pasó de 0,1097 ms a 0,1084 ms, la detección de idioma de 2,8185 ms a 2,8230 ms, la predicción neuronal de 0,3101 ms a 0,3190 ms y el análisis combinado en caliente de 3,4187 ms a 3,3611 ms. Las variaciones de hasta el 3 % se consideran ruido de medición, ya que no se modificaron reglas, pesos ni modelos.

Dataset de ejemplo

Para probar el arranque del clasificador sin depender de información personal, el repositorio incluye un dataset sintético mínimo generado bajo demanda: cinco ejemplos de phishing y cinco legítimos por idioma (español e inglés). Los ejemplos cubren urgencia, solicitud de credenciales, enlaces sospechosos y mensajes legítimos de confirmación, reuniones o boletines. No pretende ser un corpus de producción; sirve para demostrar el flujo y mantener una ejecución reproducible cuando todavía no se ha cargado un CSV real.

Resultados del clasificador neuronal

La evaluación cuantitativa de un modelo entrenado debe realizarse desde la vista de Entrenamiento con un CSV de prueba independiente. El sistema separa los datos de entrenamiento y prueba cuando el usuario proporciona ambos conjuntos y muestra accuracy, precisión, recall, F1, accuracy balanceada y la matriz VP/VN/FP/FN. Los ejemplos sintéticos incluidos en el repositorio son deliberadamente pequeños y no permiten afirmar métricas generalizables; por ello, cualquier cifra debe acompañarse del tamaño, el idioma y el origen del conjunto evaluado.

Comportamiento del sistema heurístico

El sistema heurístico es deliberadamente conservador: solo clasifica como phishing probable los mensajes que superan el umbral (por defecto 45 sobre 100) acumulando varias señales.

Sobre el dataset de ejemplo, construido con mensajes sintéticos sin cabeceras reales (sin Received ni resultados SPF/DKIM/DMARC), la heurística por sí sola detecta las señales de contenido (lenguaje urgente, saludo genérico, solicitud de credenciales o enlaces sospechosos) pero rara vez alcanza el umbral. Este comportamiento es el esperado y evidencia que sus mejores resultados se obtienen con correos reales que incluyen cabeceras de autenticación y enlaces a dominios conocidos: la heurística aporta explicabilidad, mientras que la generalización la aporta el modelo neuronal, de ahí el interés del modo combinado.

Prueba funcional de la aplicación web

Se realizaron comprobaciones funcionales sobre mensajes sintéticos en español e inglés y sobre un correo legítimo de control. Las comprobaciones verificaron que el servicio devuelve los tres resultados, aplica el umbral común, selecciona el detector del idioma correspondiente y conserva las señales y explicaciones heurísticas. La suite de 47 pruebas añade casos específicos para la API, el límite de archivos EML, la caché por idioma y el manejo de entradas malformadas; no se presentan estas comprobaciones como una evaluación estadística de producción.

Discusión

Los resultados confirman el diseño como una base funcional: la heurística aporta señales comprensibles sobre contenido y cabeceras, el clasificador neuronal reconoce patrones aprendidos y el modo combinado equilibra ambas perspectivas. La principal limitación es el tamaño del conjunto sintético de demostración y la ausencia de un corpus de producción representativo, que pueden provocar sobreajuste y métricas poco generalizables. Para una evaluación rigurosa será necesario entrenar con datasets públicos amplios y recientes y comparar con métodos del estado del arte. Aun así, el prototipo demuestra la viabilidad de integrar reglas explicables y aprendizaje supervisado en una aplicación local con varias interfaces.

Conclusiones

El Trabajo Fin de Grado ha alcanzado los objetivos planteados. Se ha estudiado el phishing desde su base psicológica y su evolución reciente, se ha revisado el estado del arte en machine learning y deep learning y se ha desarrollado una aplicación web funcional que combina análisis heurístico explicable con un clasificador neuronal TF-IDF + MLP. La aplicación permite analizar texto pegado, archivos .eml y correos importados mediante Gmail, mientras que el mismo núcleo se reutiliza opcionalmente desde una API HTTP local, una extensión de Gmail y un monitor con Telegram.

Para comprobar el cumplimiento de los objetivos, se revisa a continuación el grado de consecución de cada uno de los objetivos específicos planteados en la introducción:

El primer objetivo, analizar el fenómeno del phishing, sus fases, tipologías y evolución histórica, se aborda en el marco teórico mediante modelos de fases, vectores de ataque, modalidades según personalización y técnicas emergentes contrastadas con fuentes académicas e informes de industria.

El segundo objetivo, estudiar los fundamentos psicológicos de la ingeniería social, se cubre mediante el análisis de los principios de Cialdini, los sesgos cognitivos de Kahneman y Tversky y el impacto de la inteligencia artificial generativa.

El tercer objetivo, revisar el estado del arte en técnicas de detección, se desarrolla en la sección dedicada a machine learning clásico, deep learning, transformers y enfoques híbridos, y concluye con el hueco de investigación que justifica el prototipo.

El cuarto objetivo, diseñar e implementar un sistema heurístico explicable y un clasificador TF-IDF + MLP, se materializa en el motor de señales sobre cabeceras, contenido, URLs y HTML y en el pipeline neuronal entrenable en español e inglés.

El quinto objetivo, integrar la aplicación con Gmail y proporcionar alertas mediante Telegram, se cumple mediante el acceso OAuth incorporado en las vistas Streamlit y la monitorización opcional que reutiliza el mismo servicio local de análisis.

El sexto objetivo, evaluar el comportamiento del sistema mediante pruebas unitarias, de integración y funcionales, se aborda con la suite actual de 47 casos, la evaluación del clasificador con un conjunto de prueba independiente cuando se proporciona y la comprobación de mensajes sospechosos y legítimos en español e inglés.

Entre los logros principales destacan la modularidad del código, la selección del modelo por idioma, la API local compartida, la extensión de Gmail, la integración de Gmail y Telegram, la incorporación de métricas de exactitud, precisión, recall, F1 y accuracy balanceada en la vista de entrenamiento y un benchmark reproducible de rendimiento. La carga diferida reduce el arranque de las heurísticas un 96,6 % y el de la aplicación un 76,2 % sin alterar la inferencia. El prototipo demuestra la viabilidad de un detector de phishing explicable, eficiente y extensible dentro del alcance de un Trabajo Fin de Grado.

Trabajo futuro

Como trabajo futuro se plantea ampliar la evaluación con datasets públicos recientes en español e inglés y estudiar la robustez frente a ataques adversariales. En el plano del modelo, podrá sustituirse TF-IDF + MLP por transformers o LLMs con detección explícita de autoridad, urgencia y escasez y con explicaciones en lenguaje natural. También se contempla integrar reputación de dominios, análisis dinámico de páginas y etiquetado seguro de mensajes en Gmail. Para una publicación multiusuario de la aplicación web será necesario añadir autenticación, gestión de secretos, aislamiento de sesiones y una política de protección de datos.

Glosario

API: Interfaz de operaciones que permite a un cliente local enviar datos al backend mediante un contrato HTTP; Gmail y Telegram son integraciones externas distintas.

Aplicación web local: Interfaz Streamlit que integra la presentación y el núcleo de análisis en el mismo entorno; el proyecto también ofrece API y extensión locales opcionales.

Blacklist (lista negra): Lista local de dominios o URLs considerados no confiables que el sistema consulta durante el análisis.

Business Email Compromise (BEC): Fraude que suplanta a directivos o proveedores para autorizar pagos o transferencias fraudulentas.

DKIM: Mecanismo de autenticación de correo basado en firmas criptográficas del dominio remitente.

DMARC: Política que indica a los servidores receptores cómo tratar los mensajes que no superan SPF o DKIM.

Análisis combinado: Modo que pondera la puntuación heurística y la probabilidad del clasificador neuronal.

Streamlit: Framework de Python utilizado para construir la interfaz web interactiva del prototipo.

Heurística: Conjunto de reglas basadas en conocimiento experto que evalúan indicios de phishing de forma explicable.

MLPClassifier: Perceptrón multicapa de scikit-learn: red neuronal de alimentación directa con una o más capas ocultas.

OAuth: Protocolo de autorización empleado para acceder a la API de Gmail sin compartir la contraseña.

Phishing: Técnica de ingeniería social que suplanta entidades legítimas para obtener información sensible.

Precisión: Proporción de casos clasificados como phishing que realmente lo son.

Quishing: Variante de phishing que utiliza códigos QR como vector de entrega.

Recall: Proporción de los casos de phishing reales que el modelo detecta correctamente.

Smishing: Variante de phishing mediante mensajes SMS.

TF-IDF: Técnica de vectorización de texto que pondera la frecuencia de cada término según su importancia.

Umbral (threshold): Valor de puntuación a partir del cual un correo se considera phishing probable.

Vishing: Variante de phishing mediante llamadas telefónicas.

Referencias

- AlEroud, A., & Karabatis, G. (2020). Bypassing detection of URL-based phishing attacks using generative adversarial deep neural networks. In *Proceedings of the Sixth International Workshop on Security and Privacy Analytics (IWSPA '20)* (pp. 53-60). ACM. <https://doi.org/10.1145/3375708.3380315>
- Alghenaim, M., Alkaws, G., & Barnhart, C. R. (2025). The state of the art in AI-based phishing detection: A systematic literature review. In M. A. Al-Sharafi, M. Al-Emran, M. A. Mahmoud, & I. Arpacı (Eds.), *Current and Future Trends on AI Applications* (Studies in Computational Intelligence, Vol. 1178, pp. 431–458). Springer. https://doi.org/10.1007/978-3-031-75091-5_23
- Alkhalil, Z., Hewage, C., Nawaf, L., & Khan, I. (2021). Phishing attacks: A recent comprehensive study and a new anatomy. *Frontiers in Computer Science*, 3, Article 563060. <https://doi.org/10.3389/fcomp.2021.563060>
- Altawairy, N., Al-Turaiki, I., Alotaibi, R., & Alakeel, F. (2024). Advancing phishing email detection: A comparative study of deep learning models. *Sensors*, 24(7), 2077. <https://doi.org/10.3390/s24072077>
- Anti-Phishing Working Group. (2025). Phishing Activity Trends Report – 2nd Quarter 2025. https://docs.apwg.org/reports/apwg_trends_report_q2_2025.pdf
- Cialdini, R. B. (1984). *Influence: The Psychology of Persuasion*. William Morrow.
- Federal Bureau of Investigation. (2024). 2023 Internet Crime Report. Internet Crime Complaint Center. https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf
- Fortinet. (2025). 19 types of phishing attacks with examples. <https://www.fortinet.com/resources/cyberglossary/types-of-phishing-attacks>
- GreatHorn. (2021). 3 phases of the phishing attack kill chain. <https://web.archive.org/web/20231201201455/https://www.greathorn.com/blog/what-are-the-3-phases-of-the-phishing-attack-kill-chain/>
- Hadnagy, C. (2018). *Social Engineering: The Science of Human Hacking* (2nd ed.). Wiley.
- Heiding, F., Schneier, B., Vishwanath, A., Bernstein, J., & Park, P. S. (2024). Devising and detecting phishing emails using large language models. *IEEE Access*, 12, 42131-42146. <https://doi.org/10.1109/ACCESS.2024.3375882>
- Hoxhunt. (2024). The Phishing Kill Chain: Three Steps and Five Effects. <https://hoxhunt.com/blog/three-steps-and-five-effects-of-the-phishing-attack-kill-chain>
- Hoxhunt. (2025). Phishing Trends Report 2025. <https://hoxhunt.com/guide/phishing-trends-report>
- Hutchinson, S., Zhang, Z., & Liu, Q. (2018). Detecting phishing websites with random forest. In *Machine Learning and Intelligent Communications* (pp. 470-479). Springer. https://doi.org/10.1007/978-3-030-00557-3_46
- IBM. (2025). Cost of a Data Breach Report 2025. <https://www.ibm.com/reports/data-breach>
- Imperva. (2023). What is phishing: Attack techniques & scam examples. <https://www.imperva.com/learn/application-security/phishing-attack-scam/>
- Instituto Nacional de Ciberseguridad. (2025). Balance de ciberseguridad 2024. https://www.incibe.es/sites/default/files/Comunicación_2025/Infografía_BalanceCiberseguridad_INCIBE_2024_web.pdf
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- Koide, T., Fukushima, N., Nakano, H., & Chiba, D. (2024). ChatSpamDetector: Leveraging large language models for effective phishing email detection. arXiv preprint arXiv:2402.18093. <https://arxiv.org/abs/2402.18093>
- Kroll. (2025). Q4 2024 Cyber Threat Landscape Report: CorruptQR Campaign. <https://www.kroll.com/en/reports/cyber/threat-intelligence-reports/q4-2024-threat-landscape-report-phishing>
- Kuikel, S., Pipalai, A., & Aggarwal, P. (2025). Evaluating large language models for phishing detection, self-consistency, faithfulness, and explainability. arXiv preprint arXiv:2506.13746. <https://arxiv.org/abs/2506.13746>
- Le, H., Pham, Q., Sahoo, D., & Hoi, S. C. H. (2018). URLNet: Learning a URL representation with deep learning for malicious URL detection. arXiv preprint arXiv:1802.03162. <https://arxiv.org/abs/1802.03162>
- LevelBlue. (2024). Dissecting a multi-stage phishing attack. <https://levelblue.com/blogs/security-essentials/dissecting-a-multi-stage-phishing-attack>
- Microsoft. (2025). Microsoft Digital Defense Report 2025. <https://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/>
- MITRE. (2025). ATT&CK Framework: T1566 Phishing. <https://attack.mitre.org/techniques/T1566/>
- OpenAI. (2023). GPT-4 Technical Report. <https://arxiv.org/abs/2303.08774>
- Popescul, D., & Radu, L. D. (2025). AI in phishing detection: A bibliometric review. *Frontiers in Artificial Intelligence*, 8, 1496580. <https://doi.org/10.3389/frai.2025.1496580>

- Proofpoint. (2025). What is phishing? Meaning, attack types & more. <https://www.proofpoint.com/us/threat-reference/phishing>
- Proofpoint. (2025a). The Human Factor 2025: Vol. 1 Social Engineering. <https://www.proofpoint.com/us/resources/threat-reports/human-factor-social-engineering>
- Proofpoint. (2025b). The Human Factor 2025 Vol. 2: URL Phishing. <https://www.proofpoint.com/us/resources/threat-reports/human-factor-url-phishing>
- Rojas-Galeano, S. (2024). Zero-shot spam email classification using pre-trained large language models. arXiv preprint arXiv:2405.15936. <https://arxiv.org/abs/2405.15936>
- Sánchez-Paniagua, M., Fidalgo, E., Alegre, E., & Alaiz-Rodríguez, R. (2022). Phishing websites detection using a novel multipurpose dataset and web technologies features. *Expert Systems with Applications*, 207, 118010. <https://doi.org/10.1016/j.eswa.2022.118010>
- Thakur, K., Ali, M. L., Obaidat, M. A., & Kamruzzaman, A. (2023). A systematic review on deep-learning-based phishing email detection. *Electronics*, 12(21), 4545. <https://www.mdpi.com/2079-9292/12/21/4545>
- Trad, F., & Chehab, A. (2024). Prompt engineering or fine-tuning? A case study on phishing detection with large language models. *Machine Learning and Knowledge Extraction*, 6(1), 367-384. <https://doi.org/10.3390/make6010018>
- Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157), 1124-1131. <https://doi.org/10.1126/science.185.4157.1124>
- Uddin, M. A., Mahiuddin, M., & Sarker, I. H. (2024). An explainable transformer-based model for phishing email detection: A large language model approach. arXiv preprint arXiv:2402.13871. <https://arxiv.org/abs/2402.13871>
- Verizon. (2021). The history of phishing attacks. <https://www.verizon.com/business/resources/articles/s/the-history-of-phishing/>
- Verizon. (2025). 2025 Data Breach Investigations Report. <https://www.verizon.com/business/resources/reports/2025-dbir-executive-summary.pdf>
- Wang, Y., Zhu, W., Xu, H., Qin, Z., Ren, K., & Ma, W. (2023). A large-scale pretrained deep model for phishing URL detection. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1-5). IEEE. <https://doi.org/10.1109/ICASSP49357.2023.10095719>
- Yerima, S. Y., & Alzaylaee, M. K. (2020). High accuracy phishing detection based on convolutional neural networks. In *Proceedings of the 3rd International Conference on Computer Applications & Information Security* (pp. 1-6). IEEE. <https://doi.org/10.1109/ICCAIS48893.2020.9096869>

Anexos

Este capítulo reúne el material complementario del Trabajo Fin de Grado: la estructura completa del repositorio, los fragmentos de código fuente más representativos del sistema y una guía resumida de instalación y uso. El código completo del prototipo se encuentra disponible en el repositorio GitHub asociado a este trabajo.

Anexo A: Estructura del repositorio

La siguiente estructura recoge los componentes principales que forman parte del alcance de la aplicación web y de su motor local de análisis:

Estructura de directorios del repositorio

```
TFG/
├── src/
│   ├── app.py                # Entrada principal de Streamlit
│   ├── backend_server.py     # API HTTP local
│   ├── config_app.py         # Configuración de Gmail, Telegram y modelos
│   ├── detect_app.py         # Detección desde texto, EML y Gmail
│   ├── gmail_extension_server.py # Adaptador de la extensión Gmail
│   ├── monitor_app.py        # Vista de monitorización de Gmail
│   ├── monitor_gmail.py      # Monitorización periódica opcional
│   ├── train_app.py          # Entrenamiento y evaluación
│   ├── ui_components.py      # Componentes visuales compartidos
│   └── sistema_phishing/
│       ├── analizador_email.py # Parseo y normalización de correos
│       ├── analysis_service.py  # Coordinación del análisis local
│       ├── analyzer.py          # Orquestador heurístico
│       ├── backend_service.py   # Compatibilidad con el backend
│       └── configuracion.py     # Configuración del dominio
```

— content_signals.py	# Señales de texto y adjuntos
— correo.py	# Representación interna común
— dataset.py	# Normalización de CSV
— env_loader.py	# Variables locales de entorno
— explanations.py	# Explicaciones del riesgo
— gmail_client.py	# Integración OAuth con Gmail
— gmail_monitor.py	# Lógica de monitorización
— header_signals.py	# Cabeceras y autenticación
— heurísticas.py	# Fachada heurística
— html_signals.py	# Señales HTML
— http_api.py	# Transporte HTTP y validación
— idioma.py	# Detección de idioma
— metrics.py	# Métricas y matriz de confusión
— modelo_neural.py	# Pipeline TF-IDF + MLP
— neural.py	# Fachada neuronal
— scorer.py	# Cálculo de puntuación
— signal_builder.py	# Construcción de señales
— signals.py	# Utilidades de señales
— telegram_notifier.py	# Alertas por Telegram
— url_utils.py	# Análisis de URLs
— tests/	# Suite unittest
— scripts/	# Automatización y experimentos
— requirements.txt	
— credentials.example.json	
— .env.example	

Anexo B: Fragmentos de código fuente

A continuación se reproducen los fragmentos más representativos del sistema: la fachada del análisis heurístico, el servicio común que combina heurística y red neuronal, y el núcleo del pipeline neuronal TF-IDF + MLP.

B.1. Fachada del análisis heurístico (heurísticas.py)

```
heurísticas.py
"""Fachada de heurísticas para el sistema de detección de phishing."""
from .analyzer import PhishingAnalyzer
from .correo import CorreoAnalizado
from .url_utils import extraer_urls
__all__ = ["analizar_correo", "extraer_urls"]
def analizar_correo(correo):
    """Normaliza la entrada y delega el análisis en PhishingAnalyzer."""
    correo_analizado = CorreoAnalizado.from_input(correo)
    return PhishingAnalyzer(correo_analizado).analyze()
```

B.2. Servicio común de análisis (analysis_service.py)

```
analysis_service.py (extracto actualizado)
from collections.abc import Callable
from threading import Lock
from typing import Protocol
from .analizador_email import construir_texto_para_analisis
from .heurísticas import analizar_correo
from .idioma import detectar_idioma_correo
from .modelo_neural import ModelStorage, NeuralPhishingClassifier, NeuralPhishingDetector
MODULO_HEURISTICO = "heurístico"
MODULO_NEURAL = "neural"
MODULO_COMBINADO = "combinado"
def cargar_detector_neural(config, idioma="es"):
    ruta_principal = config.model_path_en if idioma == "en" else config.model_path_es
    classifier = ModelStorage(ruta_principal).load()
    idioma_esperado = "english" if idioma == "en" else "spanish"
    if classifier is not None and getattr(classifier, "language", None) != idioma_esperado:
        classifier = None # No reutilizar silenciosamente otro idioma.
    if classifier is None:
        classifier = NeuralPhishingClassifier(language=idioma_esperado)
        classifier.fit_default()
    return NeuralPhishingDetector(classifier)
class EmailAnalysisService:
    def __init__(self, config, heuristic_analyzer=None, detector_loader=None,
        language_detector=None):
```

```

validar_configuracion(config)
self.config = config
self._heuristic_analyzer = heuristic_analyzer or analizar_correo
self._detector_loader = detector_loader or cargar_detector_neural
self._language_detector = language_detector or detectar_idioma_correo
self._detectores = {}
self._detector_lock = Lock()
# _aplicar_umbral y construir_resultado_combinado validan el contrato y los pesos.
def analyze(self, datos_email):
    if self.config.mode == MODO_HEURISTICO:
        return _aplicar_umbral(self._heuristic_analyzer(datos_email), self.config.threshold)
    resultado_neural = self._analyze_neural(datos_email)
    if self.config.mode == MODO_NEURAL:
        return _aplicar_umbral(resultado_neural, self.config.threshold)
    resultado_heur = self._heuristic_analyzer(datos_email)
    return construir_resultado_combinado(resultado_heur, resultado_neural, self.config)
def analyze_all(self, datos_email):
    resultado_heur = _aplicar_umbral(self._heuristic_analyzer(datos_email),
self.config.threshold)
    resultado_neural = _aplicar_umbral(self._analyze_neural(datos_email),
self.config.threshold)
    resultado_combinado = construir_resultado_combinado(resultado_heur, resultado_neural,
self.config)
    return {"heuristic": resultado_heur, "neural": resultado_neural, "combinado":
resultado_combinado}
def _analyze_neural(self, datos_email):
    texto = construir_texto_para_analisis(datos_email)
    idioma = self._language_detector(texto)
    detector = self._detectores.get(idioma)
    if detector is None:
        with self._detector_lock:
            detector = self._detectores.get(idioma)
            if detector is None:
                detector = self._detector_loader(self.config, idioma)
                self._detectores[idioma] = detector
    return detector.analyze(texto, datos_email.get("from", ""), datos_email.get("subject",
""))

```

B.3. Núcleo del clasificador neuronal (modelo_neural.py)

modelo_neural.py (extracto actualizado)

```

class NeuralPhishingClassifier:
    def __init__(self, language="spanish", hiperparametros=None):
        self.language = language
        hp = hiperparametros or cargar_hiperparametros_desde_env()
        self.pipeline = Pipeline([
            ("vectorizer", TfidfVectorizer(ngram_range=hp.tfidf_ngram_range, ...)),
            ("classifier", MLPClassifier(hidden_layer_sizes=hp.mlp_hidden_layer_sizes, ...)),
        ])
    def fit(self, textos, etiquetas):
        # Valida las etiquetas y entrena una ejecución nueva.
        self.pipeline.fit(textos, etiquetas)
    def predict_proba(self, texts):
        proba = self.pipeline.predict_proba(texts)
        clases = self.pipeline.named_steps["classifier"].classes_.tolist()
        indice_phishing = clases.index(1)
        return [float(fila[indice_phishing]) for fila in proba]
    def __getstate__(self):
        state = self.__dict__.copy()
        state["training_texts"] = []
        state["training_labels"] = []
        return state
class NeuralPhishingDetector:
    def __init__(self, classifier):
        self.classifier = classifier
    def analyze(self, texto, remitente="", subject=""):
        probability = self.classifier.predict_proba([texto])[0]
        return {
            "is_phishing": probability >= 0.5,
            "risk_score": round(probability * 100, 1),
            "description": "Clasificación basada en una red neuronal entrenada.",
            "from": remitente,
            "subject": subject,
        }

```


Anexo C: Guía resumida de instalación y uso

Este anexo resume los pasos necesarios para ejecutar la aplicación web y sus adaptadores locales. Streamlit se inicia con `src/app.py`; la API HTTP local escucha por defecto en `127.0.0.1:8766` y expone `/health` y `/analyze`, mientras que el servidor de la extensión Gmail utiliza `127.0.0.1:8765`. La guía ampliada, la configuración de credenciales y la solución de problemas se mantienen en el archivo README del repositorio.

Comandos de instalación y uso

```
# 1. Crear y activar un entorno virtual
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Ejecutar la aplicación web
streamlit run src/app.py

# 3b. Opcional: API HTTP local
$env:PYTHONPATH="src"; python src/backend_server.py
# 3c. Opcional: servidor de la extensión Gmail
$env:PYTHONPATH="src"; python src/gmail_extension_server.py
# 4. Ejecutar la suite de pruebas
$env:PYTHONPATH="src"; python -m unittest discover -s tests -p "test_*.py"
```

C.1. Uso del prototipo

La aplicación ofrece varias entradas: texto pegado, archivos .eml y correos importados mediante Gmail desde la vista Detección; la extensión de Gmail envía además el mensaje visible al servidor local. La API acepta un objeto JSON con asunto, remitente, cuerpo, enlaces, anclas y cabeceras, y devuelve la misma estructura de riesgo que consume la extensión y otros clientes locales.

Para cada correo, el sistema devuelve una puntuación global de riesgo, una clasificación y un desglose de señales. Se revisan autenticación SPF/DKIM/DMARC, coherencia del remitente, urgencia en el asunto, URLs sospechosas, IPs, punycode, acortadores, redirecciones, adjuntos y técnicas de ingeniería social. Las señales activadas se acompañan de explicaciones para interpretar por qué se ha clasificado un mensaje como phishing.

El clasificador neuronal asigna una probabilidad de phishing y selecciona el modelo según el idioma detectado. La interfaz Streamlit presenta conjuntamente la puntuación del modelo y las señales explicables de la heurística.

Desde la vista Monitor se pueden comprobar correos nuevos de Gmail y, si Telegram está configurado, enviar una alerta con el remitente, el asunto y la puntuación de riesgo.

Un caso de uso habitual consiste en cargar un .eml, pegar el contenido, abrir el correo en Gmail o enviarlo a la API local. El sistema procesa la entrada en el entorno local y muestra las señales detectadas, permitiendo decidir con criterio si es seguro o debe reportarse; el tiempo de respuesta depende del tamaño del mensaje y de si es necesario entrenar un modelo sintético de sustitución.

Anexo D: Lista de verificación de la validación

La siguiente lista resume las comprobaciones realizadas sobre el prototipo durante la fase de validación, tal y como se detalla en el capítulo de pruebas y resultados.

Verificado: Comprobación de la suite de 47 pruebas unitarias y de integración.

Verificado: Benchmark reproducible de arranque e inferencia con comparación antes/después.

Verificado: Normalización correcta de etiquetas phishing y correo legítimo.

Verificado: Entrenamiento del modelo TF-IDF + MLP en español e inglés.

Verificado: Evaluación con un CSV de prueba independiente y métricas documentadas.

Verificado: Cálculo de exactitud, precisión, recall y F1.

Verificado: Arranque y navegación de la aplicación Streamlit.

Verificado: Análisis desde texto pegado y archivo .eml.

Verificado: Importación autorizada de correos mediante la API de Gmail.

Verificado: Selección del modelo neuronal según el idioma del mensaje.

Verificado: Monitorización de Gmail y notificación opcional por Telegram.